

Trust-Stream Evaluation of Trust-Guided Task Allocation in Low-Altitude UAV Networks

Anonymous Authors

Department of Computer Science and Engineering, Anonymous Institution

Correspondence: to be provided after peer-review anonymization is removed.

Abstract: Trust scores are useful in a cooperative UAV mission only when they inform an operational decision. This paper studies that link at the trust-stream level. We construct five normalized evidence streams—behavior, communication, energy, task execution, and history—and train a supervised multi-head network to estimate node risk and auxiliary task-allocation actions. The executable artifact is deliberately narrower than a full multi-agent reinforcement-learning system: it uses simulator labels during offline training, freezes the model for evaluation, and does not learn from environment rewards or deployment trajectories. On 50 held-out generated seeds with 15 UAVs and a 30% malicious-node ratio, the model obtains 94.4% accuracy and a 91.8% F-score. A same-feature MLP obtains 94.7% and 92.1%, respectively; neither difference is statistically significant. A model-based task-allocation simulation yields a 90.5% success proxy for the proposed configuration and 88.3% for the MLP, but this comparison is descriptive because the simulator applies mechanism-specific allocation rules. On one short ns-3-derived trace, the proposed model achieves 80.0% accuracy and 40.0% recall, trailing three rule-based adapters in recall. The results therefore establish reproducibility and feasibility only within the generated trust-stream setting. Claims about online adaptation, packet-level robustness, communication cost, and flight deployment require independent experiments.

Keywords: UAV networks; trust management; task allocation; zero trust; reproducible simulation

1 Introduction

UAV swarms used for inspection, sensing, and emergency response rely on nodes that remain inside the mission after admission. A valid identity is therefore not sufficient evidence of trustworthy behavior. A compromised node can selectively forward traffic, falsify a task result, or alternate between benign and harmful behavior while retaining the privileges granted by its earlier history. Dynamic topology and intermittent links make those behaviors difficult to distinguish from ordinary failures [1, 2].

Trust management addresses this problem by updating a node score from recent observations. Existing schemes use fuzzy fusion, anomaly profiles, provenance, or group decisions [7, 8, 10, 23]. Their scores, however, are often evaluated only as detector outputs. In a mission, the practical question is what changes after a node becomes risky: whether it may receive a sensitive task, how much traffic it may relay, and when it should be isolated. This study examines that narrower trust-to-allocation question.

We present MTIM as a reproducible proof of concept rather than a deployment-ready zero-trust architecture. The artifact generates five trust-factor streams, fits a supervised multi-head network, and uses its risk output in a model-based task-allocation simulation. Although the architecture was motivated by parameterized multi-agent decision making, the released training procedure is not reinforcement learning: there is no sampled return, Bellman target, policy-gradient update, or online interaction with an environment. Calling the executable model MARL would therefore overstate what was implemented.

The study asks two bounded questions. First, can temporal summaries of generated trust streams distinguish constant, dormant, and on-off attackers on held-out random seeds? Second, under the allocation rules encoded in the same simulator, does trust-guided selection improve a task-success proxy? The answer to the first question is positive within the generator but no better than a same-feature MLP. The second result is suggestive rather than causal because the task simulator contains mechanism-specific rules. A short ns-3-derived trace is included to expose, rather than hide, the remaining packet-level gap.

Problem statement. This paper addresses a trust-layer decision problem for fixed identities: how should a risk estimate derived from noisy multi-factor evidence affect a subsequent task-allocation decision? The empirical scope is a generated trust-stream benchmark. It does not cover admission control, Sybil resistance, real-time distributed execution, or flight deployment.

Figure 1 gives the intended deployment architecture. Only its trust-stream classifier and allocation proxy are exercised in the released artifact. The contributions are consequently limited to the following:

- a documented generator for behavioral, communication, energy, task, and historical trust streams under three temporal attack patterns;
- a reproducible supervised multi-head model and same-feature LR/MLP comparators evaluated on disjoint training and evaluation seeds; and
- an explicit negative result: the proposed risk head does not outperform the same-feature MLP, and the available packet-trace check is too small to establish external validity.

The remainder of this paper reviews related work, defines the intended architecture and executable subset, reports the trust-stream experiments and packet-trace check, and closes with limitations.

2 Related Work

Trust evaluation for UAV networks now includes reputation models, evidence fusion, blockchain support, and learning-based designs. Yet low-altitude UAV studies rarely integrate zero-trust verification, distributed trust learning, uncertainty aggregation, and mission incentives. We review three related areas: UAV trust management, zero-trust assessment, and MARL for UAV networking and security.

2.1 Trust Management in UAV and FANET Systems

Recent surveys show that trust management has become a core security primitive in FANETs, where dynamic topology, intermittent links, and resource limits make purely identity-based protection insufficient [1–5]. Existing schemes fall into three categories: behavior-centric trust analytics, distributed or blockchain-supported trust, and learning-enhanced trust evaluation.

Behavior-centric trust evaluation widely uses fuzzy and evidence-fusion methods to combine heterogeneous observations. Benfriha et al. proposed FUBA, which uses fuzzy logic to integrate indicators such as RSSI, packet delivery ratio, delay, battery level, and humidity to identify insider threats in FANETs [8]. Li et al. developed an adaptive multi-granularity trust management scheme for UAV visual sensor security under adversarial attacks, where coarse- and fine-grained evidence is fused to improve robustness against known and unknown threats [9]. Akram et al. introduced GALTrust, which combines generative adversarial learning with type-2 fuzzy logic to improve adaptability to emerging malicious behaviors in spatial crowdsourcing drone services [10]. More recent federated-security studies also move trust assessment toward collaborative anomaly detection, including FL-based intrusion detection for FANETs, multimodal swarm anomaly detection, and communication-efficient FL for resource-constrained UAV-IoT systems [11–13]. These studies significantly enrich trust evidence modeling. Their remaining limitation for the problem considered here is not that they cannot detect insider behavior at all, but that fuzzy, profile-based, or task-specific fusion can become sensitive to distribution shifts when the malicious population is dense and attack timing is deliberately evasive.

A second line of work improves trust persistence and auditability through distributed storage or hierarchical trust exchange. Ge et al. developed a provenance-aware distributed trust model for resilient UAV networks, highlighting the value of peer-to-peer evidence tracing in hostile environments [7]. Zuo et al. proposed a hierarchical blockchain-based trust measurement method for drone clusters to address trust inheritance, cross-cluster re-evaluation, and historical trust verification under dynamic node entry and exit [6]. Akram et al. proposed TMIoT, which integrates digital twins, blockchain, and deep-learning-based intrusion detection to secure Open RAN-based Internet of Drone Things applications [17]. Closely related secure-FL frameworks have further explored malicious-node detection and trusted collaboration through blockchain-supported service provisioning, privacy-preserving IoT security, and hierarchical federated aggregation [18–21]. Related work on secure quantum federated learning in UAV-assisted wireless networks further indicates that trust-enhanced game incentives can improve participation quality and strengthen learning robustness under strategic behavior [22]. More recently, Hu et al. proposed GDM-DTM for malicious-node detection in low-altitude UAV networks by combining group decision making with dynamic trust management [23]. These schemes improve distributed credibility, trust persistence, and malicious-node detection. For our setting, the central gap is that the trust state is usually treated as an assessment output rather than as the basis for learned reward, penalty, and task-allocation decisions.

2.2 Zero-Trust and Continuous Trust Assessment

Zero trust shifts security from perimeter-based assumptions to continuous, resource-centric verification. The NIST zero-trust architecture defines this paradigm through the principle of “never trust, always verify” and replaces implicit perimeter-based trust with continuous, context-aware verification [15]. Building on this idea, Wang et al. proposed a zero-trust-driven collaborative intrusion detection framework for IoT that uses continuous trust assessment to support distributed security decisions [24]. In the UAV and 6G context, Al Ridhawi and Aloqaily integrated zero trust, digital twins, federated learning, and blockchain into a UAV-enabled 6G framework, highlighting the broader relevance of decentralized trust enforcement in highly dynamic aerial networks [25]. In the UAV authentication domain, Wang et al. introduced a context-aware trust evaluation model for malicious UAV detection during authentication, showing that trust can also be injected into the authentication stage rather than being treated purely as a post hoc reputation signal [14].

Zero trust is particularly relevant to low-altitude UAV swarms: node roles may change during flight, links may appear and disappear rapidly, and local observations may be incomplete or noisy. Trust decisions cannot be based solely on prior membership or static credentials, but must be continuously updated from current observations, device status, communication behavior, and mission context. Therefore, trust decisions should be dynamic, uncertainty-aware, and resource-conscious rather than static or binary.

These studies move trust evaluation closer to continuous verification. Nevertheless, the current zero-trust literature still focuses mainly on authentication, access control, or intrusion detection. It does not fully address how trust should evolve online in low-altitude UAV swarms when heterogeneous evidence from behavior, communication quality, task execution, energy state, and historical interactions must be fused under strict resource constraints. In particular, uncertainty-aware aggregation and decentralized incentive adaptation remain underexplored in zero-trust UAV settings.

2.3 MARL for UAV Networking and Security

MARL has recently become a prominent framework for decentralized decision making in UAV systems. A recent survey by Ekechi et al. reports that MARL has mainly been used for UAV control, formation, coverage, coordination, and cooperative optimization, rather than for trust management itself [16]. Representative examples include the MADDPG-based multi-UAV trajectory planning framework of Wang et al. for UAV-assisted mobile edge computing [26], the semi-distributed multi-agent federated reinforcement learning approach of Nie et al. for resource management in UAV-aided MEC systems [27], and the joint optimization of trajectory control, task offloading, and resource allocation in air-ground integrated networks by Alam and Moh [28]. Closely related learning-and-optimization studies in UAV-enabled federated settings have examined asynchronous scheduling, contract-based participant selection, hierarchical personalized learning, air-ground FL coordination, and energy-aware trajectory and resource management [29–34]. Together, these cited works provide evidence that distributed learning can support optimization under partial observability and dynamic coupling among UAVs. More recently, Cao et al. explored LLM-based multi-agent situation awareness for zero-trust space-air-ground integrated networks [35]. Their results further suggest that model selection in security-oriented multi-agent systems should balance capability and deployment cost; for specialized subtasks, smaller task-adapted models can be more practical than larger general-purpose models.

However, existing MARL applications in UAV networking remain largely performance driven, with objectives such as delay reduction, energy efficiency, coverage, or offloading efficiency. The reward design in these studies is rarely aligned with trustworthiness, collusion resistance, or long-horizon security incentives. MARL has been used to optimize how UAVs move, communicate, or allocate resources, but not how they collaboratively evaluate trust, calibrate trust-factor importance, and propagate trustworthy decisions under zero-trust assumptions.

Overall, prior work has made strong progress in trust modeling, distributed auditability, and UAV-oriented MARL, but three problem-specific challenges remain insufficiently addressed for low-altitude UAV trust evaluation:

- (1) **Accurate trust assessment under high malicious-node ratios:** Fuzzy, profile-based, and group-decision mechanisms can lose discrimination when the malicious population substantially shifts local trust distributions, leading to poor mission continuity.
- (2) **Temporal and multi-factor evasion:** Dormant and on-off attackers can exploit historical

trust accumulation, while multi-factor manipulation requires deciding which behavioral, communication, energy, task, and history evidence remains reliable at each update.

(3) **Closing the trust-to-action loop:** A detector that outputs only a trust or risk score does not by itself decide how task allocation, incentives, or penalties should change; this is the specific gap exposed by the comparison with the strong risk-only MLP-Trust baseline.

The present artifact addresses only part of this gap: it evaluates temporal trust features and a score-based allocation proxy. Multi-agent execution and confidence-aware aggregation remain proposed extensions.

3 System Model

We consider a low-altitude UAV swarm executing a cooperative mission in an infrastructure-scarce environment. This section separates the intended system from the subset implemented in the artifact.

3.1 Problem Definition

At each update interval, the system receives partial and noisy observations of each UAV’s behavior, communication quality, energy state, task execution, and historical reputation. The objective is to infer a trust state that reflects the UAV’s current behavior and then use that state to regulate future participation through rewards, penalties, and task-allocation preferences. Formally, for each UAV U_k , MTIM must output a global trust score $\mathcal{T}_k(t)$, a trust class $C_k(t)$, and an incentive action $\pi_{jk}(t)$ for the local Thinker Agent j that supervises U_k :

$$(\mathcal{T}_k(t), C_k(t), \pi_{jk}(t)) = \Pi_\theta(o_j(t), \mathcal{H}_k(t)), \quad (1)$$

where $o_j(t)$ is the partial observation available to agent j , $\mathcal{H}_k(t)$ is the recent trust-evidence history of node k , and Π_θ is the learned trust-incentive policy. The conceptual objective is to improve both malicious-node detection and mission continuity while respecting update-cycle resource budgets:

$$\max_{\Pi_\theta} \mathbb{E}[\lambda_1 F_1 + \lambda_2 \text{TSR} - \lambda_3 C_{\text{comm}} - \lambda_4 T_{\text{eval}}], \quad (2)$$

subject to bounded trust scores and an incentive budget. This is a conceptual objective only. The empirical model minimizes the supervised loss in Section 3.5.3; it does not optimize F_1 , TSR, communication cost, or latency jointly.

3.2 UAV Network Model

We model the low-altitude UAV network as a dynamic ad hoc network of N heterogeneous UAV nodes, $\mathcal{U} = \{U_1, U_2, \dots, U_N\}$. Each node U_i is characterized by a tuple $\langle \mathbf{p}_i(t), v_i(t), E_i(t), C_i \rangle$, where $\mathbf{p}_i(t) \in \mathbb{R}^2$ is the 2D position at time t , $v_i(t) \in [v_{\min}, v_{\max}]$ is the velocity, $E_i(t) \in [0, E_i^{\max}]$ is the residual energy, and C_i denotes the computational capacity (CPU cycles/s). Nodes may enter or leave the swarm during the mission according to operational requirements.

The communication topology is a time-varying undirected graph $G(t) = (\mathcal{U}, \mathcal{E}(t))$, where an edge $(i, j) \in \mathcal{E}(t)$ exists iff the Euclidean distance satisfies $d_{ij}(t) = \|\mathbf{p}_i(t) - \mathbf{p}_j(t)\|_2 \leq R_{\text{comm}}$, with $R_{\text{comm}} = 300$ m (Table 2). Each link (i, j) has a time-varying quality characterized by packet delivery ratio $\text{PDR}_{ij}(t)$, received signal strength $\text{RSSI}_{ij}(t)$, and one-way latency $L_{ij}(t)$. The network is *infrastructure-scarce*: no fixed base stations or ground controllers are assumed to be continuously reachable; all trust computation and decision-making must execute on-board or via peer-to-peer exchange.

3.3 Threat Model

We use a fixed-identity synthetic adversary model. It is not adaptive to the learned parameters.

Adversary capabilities. A subset of nodes $\mathcal{M} \subset \mathcal{U}$ ($|\mathcal{M}|/N \in [0.05, 0.50]$ in our experiments) is controlled by an adversary. Each malicious node $U_m \in \mathcal{M}$ can independently manipulate any subset of the five trust dimensions by:

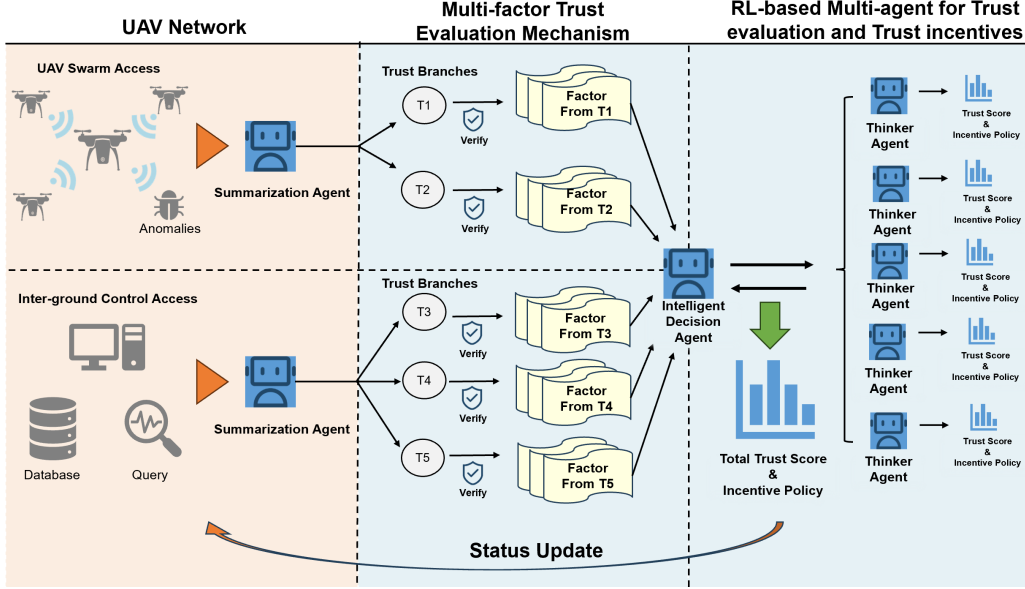


Figure 1: Intended MTIM deployment architecture. The released artifact implements the T1–T5 trust-stream features, a shared supervised network, and a model-based allocation proxy. Parallel on-board agents, confidence-weighted fusion, and a closed deployment feedback loop remain design elements rather than experimentally validated components.

- **T1 (Behavioral):** deviating from planned flight paths, skipping scheduled heartbeats, or responding inconsistently to coordination messages;
- **T2 (Communication):** performing selective forwarding (dropping a fraction $p_{\text{drop}} \in [0.3, 1.0]$ of packets), injecting spurious traffic, or manipulating transmission power to create asymmetric links;
- **T3 (Energy):** exhibiting anomalous consumption patterns (e.g., sudden depletion through jamming, or unusually low consumption by remaining idle during active mission phases);
- **T4 (Task):** returning fabricated or degraded task results (e.g., falsified sensor readings, partial payload delivery);
- **T5 (Historical):** this dimension is not directly manipulable by the adversary—it is an accumulated reputation signal maintained by MTIM—but its update is indirectly affected when the adversary manipulates T1–T4.

Attack strategies. We consider three canonical attack patterns that cover the spectrum from naive to evasive adversaries:

- Constant attack:** the malicious node exhibits adversarial behavior from $t = 0$ and maintains it throughout the mission. This models unsophisticated attackers and serves as a lower bound on detection difficulty.
- Dormant attack:** the malicious node behaves honestly during an initial warm-up period ($t \leq T_{\text{warm}}$), accumulating trust, then switches to adversarial behavior. This models adversaries that attempt to evade initial screening.
- On-off attack:** the malicious node toggles between honest and adversarial behavior with period T_{toggle} , attempting to keep its trust score near the classification boundary where decisions are most uncertain.

In mixed-attack experiments, these patterns are assigned cyclically across malicious nodes. The artifact does not implement colluding peer reports, Sybil identities, an adaptive attacker, or training-data poisoning.

Adversary knowledge. The generated attacker knows neither the fitted parameters nor the decision threshold. Because it does not respond to model outputs, the experiments measure generalization across random seeds from one generator, not robustness to a strategic white-box adversary.

Design implications. The generator mainly tests whether temporal summaries of T1 and T4 separate the three attack profiles from benign nodes. It does not test majority collusion or Byzantine aggregation. Any deployment would still require cryptographic identity binding and an independently validated observation pipeline.

3.4 Multi-factor Trust Evaluation Mechanism

MTIM represents credibility with five trust dimensions. They are not statistically independent, and the present study does not support an “exponentially harder” evasion claim. T5 is explicitly computed from the other four dimensions and is therefore a memory term, not a separate sensor.

3.4.1 Trust Factor Decomposition

T1 denotes behavioral consistency, T2 communication reliability, T3 energy behavior, and T4 task quality. T5 is an exponentially smoothed mean of T1–T4. Equations 3–6 describe how these quantities could be obtained from telemetry. In the main experiment, however, the artifact samples normalized T1–T4 values directly from attack-dependent distributions and computes T5 from those samples; it does not simulate the raw packet, position, energy, or task variables in these equations.

As illustrated in Figure 1, the trust evaluation is decomposed into multiple trust branches, each corresponding to a distinct trust factor. Let the overall trust score of UAV node i at time t be denoted as $T_i(t) \in [0, 10]$, which is a function of five fundamental trust dimensions (the formal aggregation mechanism is presented in Section 3.4.2), where each trust branch $T_i^{(m)}(t) \in [0, 1]$ is defined as follows:

- **T1: Behavioral Trust** $T_i^{(1)}(t)$. This factor evaluates operational consistency, including adherence to flight paths, communication regularity, and response reliability:

$$T_i^{(1)}(t) = \frac{1}{N_b} \sum_{j=1}^{N_b} \mathbb{I}(a_{i,j}(t) = a_{i,j}^{expected}(t)) \quad (3)$$

where $\mathbb{I}(\cdot)$ is the indicator function, $a_{i,j}(t)$ represents the actual action in behavior category j , and $a_{i,j}^{expected}(t)$ is the expected behavior. This yields $T_i^{(1)}(t) \in [0, 1]$.

- **T2: Communication Trust** $T_i^{(2)}(t)$. This factor assesses link quality and stability through packet delivery ratio, signal strength, and latency:

$$T_i^{(2)}(t) = w_p \cdot PDR_i(t) + w_s \cdot \frac{RSSI_i(t)}{RSSI_{max}} + w_l \cdot \left(1 - \frac{L_i(t)}{L_{max}}\right) \quad (4)$$

where $PDR_i(t)$, $RSSI_i(t)/RSSI_{max}$, and $1 - L_i(t)/L_{max}$ are each normalized to $[0, 1]$, and $w_p = 0.4$, $w_s = 0.3$, $w_l = 0.3$ are weighting coefficients with $w_p + w_s + w_l = 1$. Thus $T_i^{(2)}(t) \in [0, 1]$.

- **T3: Energy Trust** $T_i^{(3)}(t)$. This factor monitors energy-consumption patterns and remaining battery life to detect anomalies indicative of malfunction or compromise:

$$T_i^{(3)}(t) = \frac{E_{remaining}(t)}{E_{initial}} \cdot \exp(-\alpha \cdot |C_i(t) - \bar{C}(t)|) \quad (5)$$

where $\frac{E_{remaining}(t)}{E_{initial}} \in [0, 1]$ and the exponential term with $\alpha = 0.5$ is also bounded in $[0, 1]$, yielding $T_i^{(3)}(t) \in [0, 1]$.

- **T4: Task Execution Trust** $T_i^{(4)}(t)$. This factor measures the success rate and quality of task completion, such as sensing, data relaying, or payload delivery:

$$T_i^{(4)}(t) = \frac{1}{N_t} \sum_{k=1}^{N_t} \left(\frac{Q_{i,k}(t)}{Q_k^{max}} \cdot \mathbb{I}(\text{task_success}) \right) \quad (6)$$

where each term is normalized to $[0, 1]$, so $T_i^{(4)}(t) \in [0, 1]$.

- **T5: Historical Trust** $T_i^{(5)}(t)$. This factor incorporates long-term reputation scores and past behavior records to preserve temporal continuity in trust evaluation:

$$T_i^{(5)}(t) = \beta \cdot T_i^{(5)}(t-1) + (1-\beta) \cdot T_i^{inst}(t) \quad (7)$$

where both the historical and instantaneous components are in $[0, 1]$, and the smoothing factor $\beta = 0.7$. Thus $T_i^{(5)}(t) \in [0, 1]$.

3.4.2 Multi-factor Trust Aggregation

Each trust branch independently computes a sub-score $T_i^{(m)}(t) \in [0, 1]$ based on real-time data streams and historical logs. These sub-scores are then aggregated into a global trust score $T_i(t) \in [0, 10]$ through a weighted fusion mechanism:

$$T_i(t) = 10 \cdot \sum_{m=1}^5 \omega_m(t) \cdot T_i^{(m)}(t) \quad (8)$$

where the weights sum to one. The network learns an auxiliary weight head, but the released classifier computes risk directly from 13 summary features and does not use Eq. 8 to obtain its prediction.

3.4.3 Cross-Validation Mechanism

The multi-factor design ensures that no single point of failure or manipulation can compromise the overall trust decision. To enhance robustness, a **cross-validation mechanism** is implemented between trust branches. For any pair of trust factors m and n , the consistency metric is defined as:

$$\Gamma_{m,n}(t) = |T_i^{(m)}(t) - T_i^{(n)}(t)| \quad (9)$$

If $\Gamma_{m,n}(t) > \gamma_{th}$, where the cross-validation threshold $\gamma_{th} = 0.3$ (determined via sensitivity analysis over $\{0.1, 0.2, 0.3, 0.4, 0.5\}$ with 0.3 yielding the best F-score on a validation subset), a potential inconsistency is flagged, triggering deeper inspection. For instance, even if a UAV exhibits normal communication behavior ($T_i^{(2)}(t)$ high), but shows abnormal energy depletion ($T_i^{(3)}(t)$ low), the system can still flag it as potentially compromised. This layered verification keeps trust continuously evaluated rather than assumed from any single evidence channel.

Furthermore, the framework supports **anomaly detection** through statistical analysis of trust factor deviations:

$$\mathcal{A}_i(t) = \mathbb{I} \left(\sum_{m=1}^5 \left(\frac{T_i^{(m)}(t) - \mu_m(t)}{\sigma_m(t)} \right)^2 > \chi_{\alpha,5}^2 \right) \quad (10)$$

where $\mu_m(t)$ and $\sigma_m(t)$ are the mean and standard deviation of trust factor m across the swarm, $\chi_{\alpha,5}^2$ is the chi-squared threshold with 5 degrees of freedom at significance level $\alpha = 0.05$. This redundancy and diversity in trust evidence form the foundation for robust and adaptive trust management in UAV swarms, enabling the detection of sophisticated attacks that may manipulate multiple dimensions simultaneously.

3.5 Supervised Multi-Head Risk Model and Allocation Proxy

The intended architecture divides observation across local agents and forwards their estimates to a decision node. The executable artifact is simpler. It applies one shared supervised network independently to each node's diagnostic feature vector. Agent assignments do not enter the feature extractor, and neither confidence-weighted fusion nor budget projection is executed. We retain the broader formulation below to define a possible deployment target, but distinguish it from the evaluated model.

3.5.1 Dec-POMDP Elements

We consider a UAV swarm consisting of N UAV nodes. The trust management system employs M Thinker Agents, where each agent $j \in \{1, 2, \dots, M\}$ is responsible for a subset of UAV nodes. The

overall system can be modeled as a Dec-POMDP (decentralized partially observable Markov decision process) [37]:

$$\mathcal{G} = \langle \mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle \quad (11)$$

where $\mathcal{S}$ is the global state space, $\mathcal{O}$ is the joint observation space, $\mathcal{A}$ is the joint action space, $\mathcal{P}$ is the state transition probability, $\mathcal{R}$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor.

State Space: The state $s \in \mathcal{S}$ at time t is defined as the tuple of all UAV nodes' multi-factor trust scores:

$$s(t) = (\mathbf{T}_1(t), \mathbf{T}_2(t), \dots, \mathbf{T}_N(t)),$$

where $\mathbf{T}_k(t) = [T_k^{(1)}(t), T_k^{(2)}(t), T_k^{(3)}(t), T_k^{(4)}(t), T_k^{(5)}(t)]$ denotes the five-dimensional trust vector for node k , with each component in $[0, 1]$. The corresponding global trust score $\mathcal{T}_k(t) \in [0, 10]$ is computed by the Intelligent Decision Agent as described in Section 3.5.3.

Observation Space: Each Thinker Agent j observes a partial view of the global state, consisting only of the local trust sub-scores for the nodes in its supervised subset $\mathcal{N}_j$: $o_j(t) = (\mathbf{T}_k(t))_{k \in \mathcal{N}_j} \cup (c_{jk}(t))_{k \in \mathcal{N}_j}$, where $c_{jk}(t)$ is the confidence score assigned by agent j to its own assessment of node k .

Action Space: The action $a_j(t) \in \mathcal{A}_j$ for Thinker Agent j is a hybrid parameterized action consisting of (1) a continuous trust-weight vector $\omega_j(t) = [\omega_j^{(1)}(t), \dots, \omega_j^{(5)}(t)] \in \Delta^4$ on the probability simplex, and (2) a discrete incentive allocation vector $\pi_j(t) = [\pi_{jk}(t)]_{k \in \mathcal{N}_j}$ specifying the incentive level for each supervised node. Incentive levels are quantized into five bins $\{-2, -1, 0, 1, 2\}$, corresponding to strong penalty, weak penalty, neutral allocation, weak reward, and strong reward. Thus, trust evaluation provides the state used to judge behavior, while the learned policy decides whether the node should be rewarded with more favorable task participation or penalized through restricted access. If a sampled incentive vector violates the cycle budget $B_{\text{total}}(t)$, incentives are projected greedily by retaining actions with the largest expected utility gain per unit cost until the budget is satisfied.

Evaluated architecture: Training is centralized and supervised. Simulator labels determine the risk, action, and value targets. At inference, labels are absent, but the artifact does not emulate decentralized message exchange; it evaluates all per-node feature vectors in one process.

State Transition: The state transition $\mathcal{P}$ is defined implicitly through the trust update equations (Eqs. 25–26) in Section 3.5.4, which govern how trust scores evolve based on observations, incentives, and temporal decay.

3.5.2 Executable Multi-Head Network

The implemented policy network uses per-node diagnostic features extracted from the post-warm-up trust streams. These features include recent weakness in T1, T2, T4, and T5; low-quantile T1/T4 evidence; low-score duration; T1/T4 amplitude and standard deviation; positive degradation slope; and directional cross-factor gaps. The same feature mapping is used by MTIM and by the learned baselines introduced in Section 4.1.

For node k , let $\mathbf{x}_k \in \mathbb{R}^{13}$ denote the standardized diagnostic feature vector. A shared encoder maps $\mathbf{x}_k$ to a hidden representation h_k :

$$h_k = f_\theta(\mathbf{x}_k). \quad (12)$$

Four output heads are then computed:

$$\begin{aligned} p_k &= \sigma(g_r(h_k)), \\ \omega_k^{(m)} &= \frac{\exp(g_\omega^{(m)}(h_k))}{\sum_{\ell=1}^5 \exp(g_\omega^{(\ell)}(h_k))}, \\ \mathbf{q}_k^\pi &= g_\pi(h_k), \\ v_k &= g_v(h_k), \end{aligned} \quad (13)$$

where p_k is the malicious-risk probability, ω_k is an auxiliary trust-factor weight vector, $\mathbf{q}_k^\pi$ contains five supervised action scores, and v_k is an auxiliary service-value estimate. Only p_k is consumed by the released evaluation path; the auxiliary heads are trained and logged but do not determine the reported task-allocation probabilities. The local diagnostic trust score is

$$\hat{T}_{jk} = 10(1 - p_k), \quad (14)$$

and a node is flagged as malicious when $p_k > 0.60$. A potential deployment could select an incentive bin as

$$\pi_{jk}^{raw} = \arg \max_{\ell \in \{-2, -1, 0, 1, 2\}} q_{k,\ell}^\pi, \quad (15)$$

followed by a budget projection. That projection is not implemented in the current artifact.

3.5.3 Supervised Training Objective

Let $y_k \in \{0, 1\}$ be the simulator label used during training. Let ω_k^* be a heuristic weight target derived from the diagnostic features, π_k^* an action target derived from y_k and a risk hint, and v_k^* a service-value target derived directly from y_k . The network minimizes

$$\begin{aligned} \mathcal{L}(\theta) = & \text{BCE}_w(y_k, p_k) + 0.25 \|\omega_k - \omega_k^*\|_2^2 \\ & + 0.20 \text{CE}(\pi_k^*, \mathbf{q}_k^\pi) + 0.10 (v_k - v_k^*)^2. \end{aligned} \quad (16)$$

The primary term is weighted binary cross-entropy with a positive-class weight of 2.0. There is no reward return or temporal-difference target in Eq. 16; the value target is 0.8 for benign nodes and -0.4 for malicious nodes. The action target also depends on the ground-truth class. Training uses seeds 0–9 and a fixed decision threshold of 0.60. Evaluation seeds 1000–1049 are not used for fitting. The method is therefore a multi-task supervised classifier, not an offline or online reinforcement-learning algorithm.

The following aggregation equations specify an unevaluated extension for deployments in which several agents observe the same node. They are not invoked by the released Python evaluation.

Total Trust Score Aggregation: The Intelligent Decision Agent computes the total trust score $\mathcal{T}_k(t)$ for each UAV node k by combining the individual trust scores from all Thinker Agents that have observed or interacted with node k :

$$\mathcal{T}_k(t) = \frac{\sum_{j=1}^M \phi_{jk}(t) \cdot \hat{T}_{jk}(t)}{\sum_{j=1}^M \phi_{jk}(t)} \quad (17)$$

where $\phi_{jk}(t) \in [0, 1]$ is the confidence weight assigned to Thinker Agent j 's assessment of node k , computed based on the agent's historical accuracy:

$$\phi_{jk}(t) = \frac{\exp(-\gamma \cdot \varepsilon_{jk}(t))}{\sum_{j'=1}^M \exp(-\gamma \cdot \varepsilon_{j'k}(t))} \quad (18)$$

with $\varepsilon_{jk}(t) = \frac{1}{t} \sum_{\tau=1}^t |\hat{T}_{jk}(\tau) - T_{jk}^{obs}(\tau)|$ representing the mean absolute error of agent j for node k over time, and $\gamma = 2.0$ a scaling parameter.

Dispersion measure: A deployment may quantify disagreement among local estimates as

$$\sigma_k^2(t) = \frac{\sum_{j=1}^M \phi_{jk}(t) \left(\hat{T}_{jk}(t) - \mathcal{T}_k(t) \right)^2}{\sum_{j=1}^M \phi_{jk}(t)} + \sigma_0^2 \quad (19)$$

where $\sigma_0^2 = 0.1$ is a variance floor. This weighted dispersion is not a Bayesian posterior variance unless a probabilistic observation model is supplied.

Final Trust Classification: Based on the total trust score and uncertainty, the Intelligent Decision Agent classifies nodes into trust categories:

$$C_k(t) = \begin{cases} \text{Trusted} & \text{if } \mathcal{T}_k(t) \geq \theta_{\text{high}} \text{ and } \sigma_k(t) \leq \sigma_{\text{th}} \\ \text{Suspicious} & \text{if } \theta_{\text{low}} \leq \mathcal{T}_k(t) < \theta_{\text{high}} \text{ or } \sigma_k(t) > \sigma_{\text{th}} \\ \text{Malicious} & \text{if } \mathcal{T}_k(t) < \theta_{\text{low}} \text{ and } \sigma_k(t) \leq \sigma_{\text{th}} \end{cases} \quad (20)$$

where θ_{high} and θ_{low} are trust thresholds, and σ_{th} is the uncertainty threshold.

A future decision node could map global trust assessments to resource-allocation actions:

$$\mathbf{\Pi}(t) = \{\pi_{jk}(t) : j \in [1, M], k \in \mathcal{N}_j\}, \quad (21)$$

where $\pi_{jk}(t) \in \mathcal{A}_{\text{inc}} = \{-2, -1, 0, 1, 2\}$ is the incentive action for node k by agent j , encoding strong penalty (-2) through strong reward ($+2$). The policy is determined by solving a constrained optimization that maximizes expected network-wide utility:

$$\max_{\Pi(t)} \mathbb{E} \left[\sum_{k=1}^N U_k(\mathcal{T}_k(t), \pi_{\cdot k}(t)) \right], \quad (22)$$

subject to the budget constraint $\sum_{j=1}^M \sum_{k \in \mathcal{N}_j} c(\pi_{jk}(t)) \leq B_{\text{total}}(t)$, where $c(\pi) = |\pi|$ quantifies the resource cost of each incentive action. The per-node utility function is:

$$U_k(\mathcal{T}_k(t), \pi_{\cdot k}(t)) = \mathcal{T}_k(t) \cdot (1 + \delta \cdot \mathbb{I}(\pi_{\cdot k}(t) > 0)) - \kappa \cdot \mathbb{I}(\mathcal{T}_k(t) < \theta_{\text{low}}), \quad (23)$$

with $\delta = 0.2$ and $\kappa = 2.0$. These equations are design specifications. The current task-success simulator instead converts each method's score to a sampling weight and applies a stronger down-weighting rule to MTIM; it does not solve Eq. 22.

3.5.4 State Update Mechanism for Trust Evolution

The following state update is a proposed deployment mechanism, not the recurrence used in the main generator. In the artifact, T5 follows $T_5(t) = 0.84T_5(t-1) + 0.16\bar{T}_{1:4}(t)$ and the trained risk model is evaluated on summary windows. Incentive outputs do not alter later trust streams. A possible deployment transition is

$$\mathbf{T}(t+1) = f(\mathbf{T}(t), \mathbf{A}^{\text{inc}}(t), \mathbf{A}^{\text{task}}(t), \mathbf{E}(t)) \quad (24)$$

where $\mathbf{T}(t) \in \mathbb{R}^N$ is the vector of global trust scores, $\mathbf{A}^{\text{inc}}(t)$ is the incentive action matrix, $\mathbf{A}^{\text{task}}(t)$ captures task execution outcomes, and $\mathbf{E}(t)$ represents environmental factors. The trust score for each node k evolves according to the following three-stage process:

Temporal Trust Decay: To prevent trust stagnation and enforce continuous verification, we incorporate a temporal decay factor:

$$\tilde{\mathcal{T}}_k(t+1) = \mathcal{T}_k(t) \cdot \exp(-\rho \cdot \Delta t) \quad (25)$$

where $\rho = 0.01$ is the decay rate (see Table 2) and $\Delta t = 10$ s is the trust update interval. This ensures that trust gradually diminishes without positive reinforcement.

Trust Update Based on Observations: The final trust score is updated via a Bayesian filter that combines the decayed prior with new observations:

$$\begin{aligned} \mathcal{T}_k(t+1) &= a_k(t) \tilde{\mathcal{T}}_k(t+1) + (1 - a_k(t)) \\ &\quad \cdot T_k^{\text{obs}}(t+1), \\ a_k(t) &= \frac{\sigma_{\text{obs}}^2}{\sigma_k^2(t) + \sigma_{\text{obs}}^2}. \end{aligned} \quad (26)$$

where $\sigma_{\text{obs}}^2 = 0.5$ is the observation noise variance, and $T_k^{\text{obs}}(t+1)$ is the aggregated observed trust score from all Thinker Agents. This form assigns more weight to new observations when the prior uncertainty $\sigma_k^2(t)$ is high, and more weight to the decayed prior when the current estimate is confident. The observation term is:

$$T_k^{\text{obs}}(t+1) = \frac{1}{|\mathcal{M}_k(t+1)|} \sum_{j \in \mathcal{M}_k(t+1)} T_{jk}^{\text{obs}}(t+1) \quad (27)$$

with $\mathcal{M}_k(t+1)$ denoting the set of agents that observed node k at time $t+1$.

Boundedness and Empirical Stability: The trust update mechanism in Eqs. 25–26 admits bounded updates under the $[0, 10]$ trust scale. Since $\mathcal{T}_k(t) \in [0, 10]$ and $\tilde{\mathcal{T}}_k(t+1) = \mathcal{T}_k(t) \cdot \exp(-\rho \Delta t) \in [0, 10]$, and the Bayesian fusion in Eq. 26 is a convex combination of two bounded terms (with coefficients in $[0, 1]$ that sum to one), the trust score remains in $[0, 10]$ for all t , preventing numerical divergence.

We do not claim convergence of the Dec-POMDP or of the generated trajectories. The boundedness argument applies only to the proposed convex update.

Scope of the update claim: Equations 25–26 show boundedness of the proposed update only. They do not establish convergence, and the released simulator does not close the action-to-future-state loop.

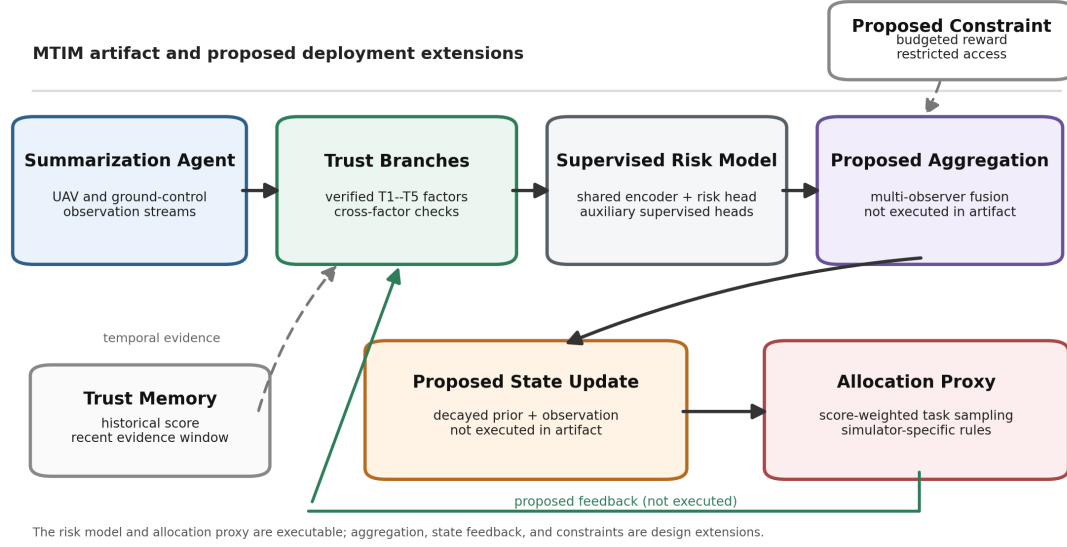


Figure 2: Proposed state-update and allocation loop. Solidly implemented elements in the artifact are trust-stream generation, feature extraction, risk inference, and a task-allocation proxy. Confidence fusion, budgeted incentives, and action-dependent future trust remain unvalidated extensions.

3.5.5 Evaluation Metrics

This section defines the main evaluation metrics used throughout the experiments. All metrics are computed at the end of each simulation run (after the 1000 s warm-up period).

Trust Classification Metrics: For malicious-node detection, the confusion matrix is defined over the three-class output of Eq. 20, with Suspicious mapped to Malicious for binary evaluation. Let TP, TN, FP, FN denote true/false positives/negatives, respectively. Detection accuracy, precision, recall, and F_1 -score are:

$$\begin{aligned} \text{Accuracy} &= \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, & \text{Precision} &= \frac{\text{TP}}{\text{TP} + \text{FP}}, \\ \text{Recall} &= \frac{\text{TP}}{\text{TP} + \text{FN}}, & F_1 &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \end{aligned} \quad (28)$$

Ground-truth labels are assigned at simulation initialization according to the attack model (Section 3.3) and are never observable by the Thinker Agents during execution.

Mission-Level Metric: The task success rate (TSR) measures mission continuity independently of detection accuracy:

$$\text{TSR} = N_{\text{succ}} / N_{\text{all}}, \quad (29)$$

In the artifact, each trial samples 220 single-node assignments. A benign selection succeeds with base probability 0.90 and a malicious selection with 0.30, followed by a small score adjustment. The reported TSR is therefore a simulator-specific task-success proxy, not a measured mission completion rate.

Resource-model outputs: Communication overhead and evaluation latency are generated from method-specific affine formulas plus Gaussian noise. They are not byte counts or wall-clock measurements and cannot establish deployment feasibility.

3.5.6 Algorithm Summary

Table 1 consolidates the MTIM trust evaluation and incentive loop for a single update cycle.

4 Performance Evaluation

Our evaluation targets four questions:

Table 1: MTIM trust evaluation and incentive cycle for one update interval Δt .

Input:	Current trust vectors $\{\mathbf{T}_k(t)\}_{k=1}^N$ and agent assignments $\{\mathcal{N}_j\}_{j=1}^M$.
Output:	Global trust scores $\{\hat{T}_k(t)\}_{k=1}^N$, classifications $\{C_k(t)\}_{k=1}^N$, and incentives $\{\pi_{jk}(t)\}$.

- 1 For each Thinker Agent $j = 1, \dots, M$, collect local observations $o_j(t) = \{\mathbf{T}_k(t)\}_{k \in \mathcal{N}_j}$.
- 2 Extract diagnostic features and compute p_k and auxiliary heads using the frozen network in Eq. 13.
- 3 Estimate local trust as $\hat{T}_{jk}(t) = 10(1 - p_k)$ and retain $\omega_j(t)$ for factor-level interpretability.
- 4 Apply the cross-validation check in Eq. 9 and flag inconsistent nodes.
- 5 Classify nodes at the fixed risk threshold $p_k > 0.60$.
- 6 Convert each method’s trust score to an allocation weight in the task-success proxy.
- 7 Sample tasks and log classification and proxy outcomes.
- 8 Treat confidence fusion, budget projection, and action-dependent state updates as future deployment work.

(Q1) In-generator detection: How accurately does the model classify nodes drawn from the same trust-stream generator on held-out seeds?

(Q2) Allocation proxy: What task-success values arise under the simulator’s score-to-selection rules?

(Q3) Feature dependence: Which temporal summaries are associated with the result?

(Q4) External check: Does the ranking persist on the bundled ns-3-derived trace?

4.1 Simulation Setup

Implementation. The evaluation uses a reproducible trust-stream generator and a supervised PyTorch network. All methods consume the same normalized T1–T5 arrays. FUBA, GALTrust, and GDM-DTM are local adapters rather than the original implementations. A separate converter is exercised on one ns-3-derived trace.

Network and mobility. The main generator runs for 500 steps at a nominal 10 s interval. It samples T1–T4 directly from fixed benign and attack-dependent means, adds Gaussian noise and node bias, and updates T5 recursively. Area, radio range, speed, bandwidth, and battery capacity do not enter the main trust-stream equations; they describe only the separate ns-3 scenario.

Adversary configuration. Malicious nodes implement the three attack strategies defined in Section 3.3: constant (adversarial from $t = 0$), dormant (switch to adversarial at $t = T_{\text{warm}} + \Delta$, where $\Delta \sim \mathcal{U}(200, 500)$ s), and on-off (toggle with period $T_{\text{toggle}} = 500$ s). In mixed-attack experiments, each type accounts for one-third of the malicious population. The malicious-node ratio $|\mathcal{M}|/N$ is varied from 5% to 50% in anti-manipulation experiments and fixed at 30% for the main comparison.

Training protocol. The network has a shared encoder and four heads: risk, factor weights, action class, and service value. All targets are supervised or heuristic. Seeds 0–9 produce 450 node-level training samples; evaluation uses seeds 1000–1049. The fixed threshold is 0.60. Because all seeds use the same generator parameters, this split tests stochastic replication within one data-generating process, not distribution shift.

Evaluation protocol. Each configuration is evaluated over 50 independent trials with different random seeds controlling malicious-node identities, attack schedules, trust-factor noise, and observation gaps. Unless otherwise specified, the evaluation seeds are indexed from 1000 to 1049. All reported results are computed from the trained execution policy and fixed baseline thresholds, and are reported as mean $\pm$ standard deviation.

Baselines. We compare MTIM against six methods spanning the design spectrum from naive fusion to learned trust classification:

- **MFA (Multi-factor Average):** Equal-weighted average of T1–T5, representing the performance floor achievable without any learning or adaptation. Included to isolate the gain attributable to adaptive weighting.

Table 2: Executable trust-stream and training parameters.

Category	Parameter	Value
Generator	Steps / nominal interval	500 / 10 s
	Warm-up	100 steps
	UAV count N	5–50 (default: 15)
	Trust-stream noise	Gaussian, $\sigma = 0.10$
Attacks	Types	constant, dormant, on–off
	Main malicious ratio	30%
	Ratio grid	5–50%
Training	Network	PyTorch supervised multi-head MLP
	Training seeds	0–9
	Evaluation seeds	1000–1049
	Decision threshold	0.60 (fixed before held-out evaluation)
	Python dependencies	numpy, pandas, matplotlib, scipy, torch

- **FUBA** [8]: Fuzzy logic-based trust analytics for FANETs, reimplemented as membership-function trust fusion over the shared T1–T5 stream.
- **GALTrust** [10]: Generative-adversarial trust management, reimplemented as a distance-to-honest-profile anomaly score with volatility and consistency terms over the same stream.
- **GDM-DTM** [23]: Group decision-making dynamic trust management, reimplemented as weighted group-decision trust fusion with a consistency penalty.
- **LR-Trust**: A logistic risk classifier trained on exactly the same diagnostic features and training seeds as MTIM, but without MTIM’s trust-weight, incentive, or value heads.
- **MLP-Trust**: A two-layer neural risk classifier trained on exactly the same diagnostic features and training seeds as MTIM, but without MTIM’s trust-weight, incentive, or value heads.

All baselines share the identical generated trust-factor streams and ground-truth labels as MTIM. Because official source code and original feature pipelines for FUBA, GALTrust, and GDM-DTM are not bundled, those baseline results should be interpreted as paper-level reimplementations under a common T1–T5 interface, not as definitive re-evaluations of the original authors’ implementations. LR-Trust and MLP-Trust are added specifically to test whether MTIM’s advantage persists against stronger learned classifiers with identical input information.

Comparison and statistical protocol. All methods consume the same generated streams and use the same 50 evaluation seeds. MTIM, LR-Trust, and MLP-Trust use the same feature vectors. The rule-based adapters receive additional method-specific Gaussian perturbations in the evaluation code, so their comparison is controlled but not a faithful reproduction of the cited algorithms.

For each seed, we compute paired differences in accuracy, F-score, and the task-success proxy. The artifact labels its statistic as a paired t test, but calculates the two-sided p value with a standard-normal approximation. We apply Holm correction across 18 comparisons. Accuracy and F-score do not differ significantly between MTIM and MLP-Trust. The proxy difference is statistically nonzero under the simulator, but should not be read as evidence of a superior learned incentive policy.

TSR is a model-based proxy. The cost and time columns are outputs of affine formulas with injected noise; they are not measurements. Under Holm correction, MTIM’s Accuracy and F-score differ from the rule adapters and LR-Trust, but not from MLP-Trust.

Analysis of Table 3. MTIM achieves 94.4% accuracy, a 91.8% F-score, and a 90.5% task-success proxy. Three observations delimit the result:

(i) **Temporal features fit the generator.** MFA reaches 80.3% accuracy, FUBA 84.1%, and GDM-DTM 85.1%. The learned models exploit low-quantile and slope features that closely match how dormant and on–off attacks are generated. This is an in-distribution advantage, not evidence of robustness to unseen attack mechanisms.

Table 3: Performance comparison of MTIM with baseline methods under the unified experimental setting (30% malicious-node ratio, 15 UAVs, 5000 s simulation). All values are mean $\pm$ standard deviation over 50 independent trials.

Method	Accuracy	F-score	TSR proxy	Cost model (KB/UAV)	Time model (ms)
MFA	80.3% $\pm$ 10.5%	69.4% $\pm$ 16.3%	79.7% $\pm$ 4.8%	11.3 $\pm$ 1.8	10.2 $\pm$ 4.5
FUBA [8]	84.1% $\pm$ 9.2%	76.1% $\pm$ 14.6%	82.5% $\pm$ 4.2%	16.7 $\pm$ 1.7	17.1 $\pm$ 4.1
GALTrust [10]	76.1% $\pm$ 6.0%	41.7% $\pm$ 21.8%	75.7% $\pm$ 4.2%	31.4 $\pm$ 1.8	40.6 $\pm$ 4.0
GDM-DTM [23]	85.1% $\pm$ 8.3%	77.3% $\pm$ 12.6%	81.3% $\pm$ 3.9%	19.4 $\pm$ 1.7	21.5 $\pm$ 3.7
LR-Trust	88.1% $\pm$ 9.6%	81.5% $\pm$ 15.4%	87.6% $\pm$ 3.6%	21.3 $\pm$ 1.8	24.5 $\pm$ 3.7
MLP-Trust	94.7% $\pm$ 5.6%	92.1% $\pm$ 7.9%	<u>88.3%</u> $\pm$ 3.9%	25.2 $\pm$ 2.0	36.1 $\pm$ 3.6
MTIM (ours)	<u>94.4%</u> $\pm$ 5.6%	<u>91.8%</u> $\pm$ 8.0%	90.5% $\pm$ 3.7%	34.0 $\pm$ 1.8	58.3 $\pm$ 4.3

(ii) **Strong learned baselines narrow the detection claim.** LR-Trust improves over non-neural fusion baselines but still trails MTIM by 6.3 pp in accuracy. MLP-Trust reaches 94.7% accuracy and 92.1% F-score, which are statistically indistinguishable from MTIM’s 94.4% and 91.8%. Thus, the evidence does not support claiming that MTIM is a strictly better detector than a strong neural risk classifier with the same features; the stronger claim is that MTIM links comparable detection quality to better mission-level action.

(iii) **The allocation result is confounded by simulator rules.** MTIM’s proxy is 2.2 pp above MLP-Trust, but the task simulator down-weights nodes predicted malicious more strongly for MTIM and adds 0.02 to benign MTIM assignments. The reported incentive head is not used in this calculation. The difference therefore cannot be attributed to learned incentive coupling.

Overhead trade-off.

The values in the last two columns are generated by `resource_metrics`; no packets are serialized and no inference path is timed. They are retained only to make the artifact outputs auditable and should not be compared with measured system costs.

Robustness across malicious-node ratios. Table 4 reports accuracy and TSR at 10%, 30%, and 50% malicious-node ratios. MTIM retains the best TSR across all three ratios. For detection accuracy, MTIM and MLP-Trust are closely matched: MLP-Trust is slightly higher at 10% and 30%, whereas MTIM is higher at 50%. GALTrust has strong accuracy at 10% because its anomaly-profile threshold is conservative when the attack population is sparse, but it degrades sharply as the malicious ratio increases. MTIM’s accuracy remains between 94.4% and 96.8%, while TSR remains above 88%, indicating that the trained policy continues to support mission-level decisions under dense adversarial pressure. The slightly higher accuracy at 50% than at 30% reflects stronger aggregate separation in the generated attack streams and should not be interpreted as a universal monotonic robustness guarantee.

Table 4: Detection accuracy and TSR at three malicious-node ratios ($N = 15$, 50 trials). Best values in **bold**, second-best underlined.

Method	10% Malicious		30% Malicious		50% Malicious	
	Accuracy	TSR	Accuracy	TSR	Accuracy	TSR
MFA	86.1 $\pm$ 9.5	87.4 $\pm$ 3.1	80.3 $\pm$ 10.5	79.7 $\pm$ 4.8	76.9 $\pm$ 8.2	71.1 $\pm$ 4.9
FUBA	83.7 $\pm$ 9.8	87.8 $\pm$ 3.3	84.1 $\pm$ 9.2	82.5 $\pm$ 4.2	81.1 $\pm$ 9.7	73.8 $\pm$ 5.2
GALTrust	<u>91.2</u> $\pm$ 4.4	85.2 $\pm$ 3.2	76.1 $\pm$ 6.0	75.7 $\pm$ 4.2	63.7 $\pm$ 7.6	65.3 $\pm$ 3.8
GDM-DTM	87.2 $\pm$ 7.9	88.5 $\pm$ 2.3	85.1 $\pm$ 8.3	81.3 $\pm$ 3.9	82.4 $\pm$ 7.8	73.9 $\pm$ 4.5
LR-Trust	90.4 $\pm$ 7.4	90.2 $\pm$ 2.0	88.1 $\pm$ 9.6	87.6 $\pm$ 3.6	88.1 $\pm$ 8.1	85.3 $\pm$ 4.6
MLP-Trust	97.3 $\pm$ 4.3	90.6 $\pm$ 2.0	94.7 $\pm$ 5.6	88.3 $\pm$ 3.9	94.4 $\pm$ 5.1	85.6 $\pm$ 5.3
MTIM (ours)	<u>96.8</u> $\pm$ 4.5	92.6 $\pm$ 1.9	<u>94.4</u> $\pm$ 5.6	90.5 $\pm$ 3.7	95.5 $\pm$ 4.4	88.8 $\pm$ 4.6

Policy fitting stability. On training seeds 0–9, the final MTIM policy reaches 100.0% training accuracy and F-score at threshold 0.60, with mean malicious risk 0.9997 and mean benign risk 0.0004. This complete training-set separation indicates that the generated training streams are highly learnable, so the reported performance is based on held-out seeds 1000–1049 rather than training metrics. The

learned trust-weight head assigns the largest average weights to T4 (0.323) and T1 (0.314), followed by T2 (0.145), T5 (0.137), and T3 (0.081), matching the threat model in which sustained task/behavior degradation is harder for evasive attackers to hide than energy behavior.

4.2 Trust Evolution Analysis

Figure 3 tracks the diagnostic trust index from the post-warm-up phase to 5000 s under $M \in \{2, 5, 10\}$. The curves are recomputed by repeatedly truncating each generated stream and applying the same fitted model. Because M affects the generator’s random seed but does not enter the policy features, these curves do not isolate an agent-count effect.

Benign nodes remain mostly above 8.0, while malicious nodes decline as low-quantile and slope evidence accumulates. Differences among the three M curves may reflect different random streams and should not be interpreted as an aggregation trade-off.

Attack-pattern sensitivity. Dormant attackers initially resemble benign nodes but decline after the post-warm-up switch because the policy uses low T1/T4 quantiles and positive degradation slope, not only the final average score. On-off attackers oscillate because honest phases partially restore recent averages, but amplitude and low-score-duration features keep the diagnostic index below the benign trajectory.

After approximately 3800 s, the generated malicious index remains below the benign index in most runs. This descriptive separation is not a convergence result.

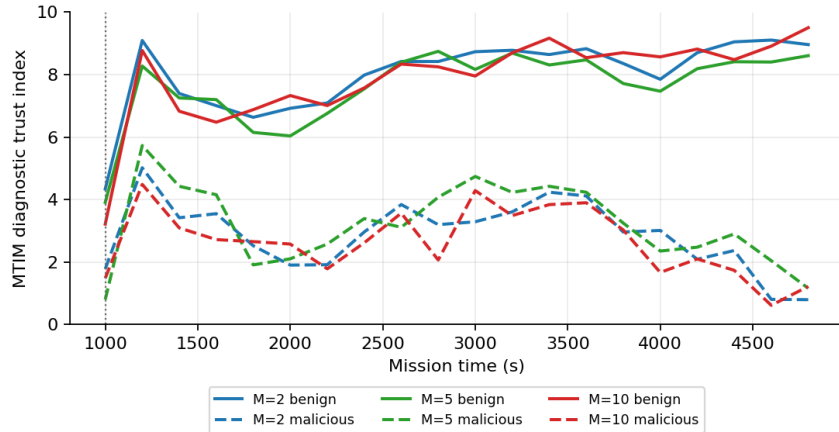


Figure 3: Diagnostic trust index on generated streams for three seed conditions labeled by M . Since M is not a policy feature, the curves do not identify a causal agent-count effect. Lower values indicate higher predicted risk.

4.3 Comparative Performance Analysis

Anti-manipulation under varying malicious ratios (Figure 4(a)). MTIM maintains high accuracy for malicious ratios of 20% and above, with 94.4% accuracy at 30% and 95.5% at 50%. MLP-Trust is a strong detection competitor under the same features, confirming that the diagnostic feature set itself carries substantial discriminative information. GALTrust is highly accurate at 5–10% malicious ratios but falls to 63.7% at 50%, indicating that the profile-distance adapter becomes unstable when the attack population substantially shifts the empirical trust distribution. MTIM does not show monotonic degradation across this generated ratio grid; this is favorable for the bounded setting but should be interpreted cautiously because stronger collusion and adaptive poisoning are not modeled.

Trust dynamics under attack patterns (Figure 4(b)). Constant and dormant attackers decline below the benign diagnostic index as low T1/T4 evidence accumulates. Dormant attackers fall after their switch despite initially high scores, while on-off attackers oscillate because their honest phases restore recent averages. This behavior supports a conservative interpretation: MTIM identifies many on-off

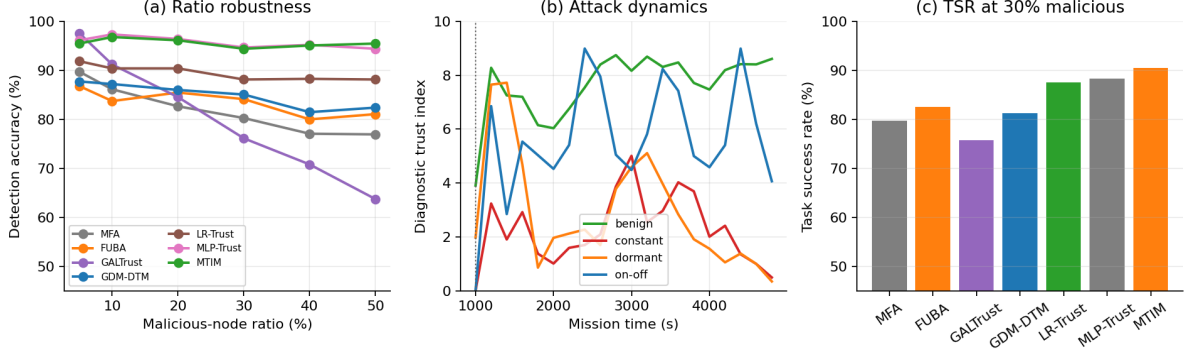


Figure 4: Results from the generated trust-stream environment. (a) Detection accuracy across malicious-node ratios. (b) Diagnostic index for the generator’s four behavior profiles. (c) Model-based task-success proxy at a 30% malicious-node ratio.

attackers as high risk, but their oscillatory trajectory remains the hardest case and drives the wide F-score variance in Table 3.

Task success comparison (Figure 4(c)). MTIM has the highest simulated proxy (90.5%), followed by MLP-Trust (88.3%) and LR-Trust (87.6%). This ordering is partly built into the allocation function, which applies MTIM-specific multipliers. It is not evidence that the auxiliary incentive head learned a better policy.

Stress cases not claimed. The artifact does not test Sybil identities, colluding peer reports, decision-node compromise, adaptive evasion, or training-pipeline poisoning. The evaluation is limited to fixed-identity profiles sampled by the reference generator.

ns-3 trace-driven sanity check. To connect the trust-stream interface to packet/mobility observations, we converted the bundled 500 s ns-3-derived trace into T1–T5 streams and evaluated all methods on the converted trace. This is a single seed-1000 trace with 15 UAVs, 49 update steps, 10 benign nodes, three constant attackers, and two on-off attackers; it is not a multi-seed packet-level validation. Table 5 shows that MTIM correctly preserves all benign nodes but detects only two of five malicious nodes on this short trace. In this trace-only setting, MFA, FUBA, and GDM-DTM detect three malicious nodes. Therefore, the ns-3 trace path should be interpreted as an interface-validation sanity check and as evidence that packet-level validation can expose harder short-horizon behavior than the main trust-stream simulator.

Table 5: Trace-driven validation on the bundled 500 s ns-3-derived trust stream (single seed, 15 UAVs, 49 update steps).

Method	Accuracy	Precision	Recall	F-score	TP/TN/FP/FN	Mean score (B/M)
MFA	86.7%	100.0%	60.0%	75.0%	3/10/0/2	0.900/0.619
FUBA	86.7%	100.0%	60.0%	75.0%	3/10/0/2	0.939/0.574
GALTrust	66.7%	50.0%	60.0%	54.5%	3/7/3/2	0.718/0.428
GDM-DTM	86.7%	100.0%	60.0%	75.0%	3/10/0/2	0.938/0.586
LR-Trust	80.0%	100.0%	40.0%	57.1%	2/10/0/3	1.000/0.600
MLP-Trust	80.0%	100.0%	40.0%	57.1%	2/10/0/3	1.000/0.584
MTIM (ours)	80.0%	100.0%	40.0%	57.1%	2/10/0/3	1.000/0.482

4.4 Ablation Study

Table 6 reports code-level variants. These are not clean single-component ablations: several rows replace the trained model with a hand-coded scoring rule, add noise, or subtract a fixed 0.09 from the task-success proxy. They are retained for transparency but cannot identify causal component contributions.

Table 6: Ablation study of MTIM components (30% malicious-node ratio, 15 UAVs). All values are mean $\pm$ std over 50 trials.

Code-level variant	Accuracy	F-score	TSR proxy
MTIM (full)	94.4% $\pm$ 5.6%	91.8% $\pm$ 7.9%	90.5% $\pm$ 3.7%
w/o Cross-factor Checking	92.9% $\pm$ 5.9%	89.5% $\pm$ 8.6%	89.5% $\pm$ 4.2%
w/o Bayesian Aggregation	84.1% $\pm$ 6.1%	67.7% $\pm$ 16.2%	81.9% $\pm$ 4.1%
w/o Temporal Decay	79.5% $\pm$ 6.1%	52.8% $\pm$ 20.5%	80.0% $\pm$ 3.8%
w/o Adaptive Policy	82.3% $\pm$ 7.0%	64.2% $\pm$ 18.2%	80.9% $\pm$ 4.5%
w/o Incentive Mechanism	94.3% $\pm$ 5.7%	91.7% $\pm$ 8.0%	81.4% $\pm$ 3.7%
T1 Only	80.8% $\pm$ 11.5%	74.9% $\pm$ 14.2%	82.6% $\pm$ 4.5%
T2 Only	63.7% $\pm$ 11.7%	53.3% $\pm$ 13.8%	76.2% $\pm$ 5.1%
T3 Only	57.6% $\pm$ 12.2%	56.0% $\pm$ 9.8%	75.9% $\pm$ 6.1%
T4 Only	82.3% $\pm$ 10.1%	76.5% $\pm$ 12.9%	83.2% $\pm$ 4.3%
T5 Only ^b	84.0% $\pm$ 9.7%	79.2% $\pm$ 13.1%	82.6% $\pm$ 3.9%

^b T5-only applies exponential smoothing to T1–T4 without a learned model. It is one hand-coded temporal baseline, not an upper bound on temporal smoothing.

Analysis of Table 6. Only the cross-factor row resembles a feature ablation, and it also adds Gaussian noise. The “Bayesian aggregation,” “temporal decay,” and “adaptive policy” rows are alternative hand-coded classifiers rather than the full model with one component removed. The “without incentive” row perturbs risk and deducts 0.09 from TSR by construction. Consequently, the table is diagnostic of code paths, not evidence that any named architectural component causes the reported change.

Single-branch baselines (T1–T5 only). Single-branch variants show that T4-only (82.3%) and T5-only (84.0%) are strong but insufficient; both trail MTIM by at least 10.4 pp in accuracy and by more than 7 pp in TSR. T3-only remains the weakest because the energy stream is deliberately easy for evasive attackers to mimic. These results support multi-factor temporal fusion, while also revealing that the current model places most discriminative power in behavior/task-history channels.

4.5 Scalability and Sensitivity Analysis

We also vary the number of generated nodes from 5 to 50. Accuracy is empirical; time and communication values are formula outputs rather than measurements.

Table 7: Generated-node-count sensitivity under a 30% malicious-node ratio. Time and communication columns are formula outputs, not measurements.

UAV count N	Seed parameter M	Accuracy (%)	Time model (ms)	Cost model (KB/UAV)
5	2	97.2 $\pm$ 7.0	36.5 $\pm$ 3.0	27.1 $\pm$ 1.9
15	5	94.4 $\pm$ 5.6	58.3 $\pm$ 4.3	34.0 $\pm$ 1.8
30	10	93.5 $\pm$ 3.8	95.1 $\pm$ 4.2	44.9 $\pm$ 1.9
50	17	95.2 $\pm$ 2.9	152.0 $\pm$ 3.8	59.8 $\pm$ 1.7

Detection accuracy stays above 93% on the generated size grid. The apparent growth in time and communication is predetermined by `resource_metrics`; no scaling law can be inferred from those columns.

Sensitivity analysis is limited to the fitted architecture, fixed threshold, and one trust-stream generator. Loss weights, temporal windows, threshold calibration, and distribution shift have not been studied systematically.

4.6 Trust Factor Correlation Analysis

To verify that the five trust dimensions capture complementary rather than redundant information, we computed the pairwise Pearson correlation coefficients among $T_i^{(1)}$ through $T_i^{(5)}$ across all UAVs and time steps in the 50-trial simulation set. The correlation matrix (averaged over trials) is:

Table 8: Pairwise Pearson correlations among the five trust dimensions (mean $\pm$ std over 50 trials).

	T1 (Behavior)	T2 (Comm.)	T3 (Energy)	T4 (Task)	T5 (History)
T1	1.00	0.28 ± 0.10	0.11 ± 0.08	0.41 ± 0.10	0.64 ± 0.08
T2	—	1.00	0.13 ± 0.09	0.27 ± 0.10	0.51 ± 0.07
T3	—	—	1.00	0.11 ± 0.09	0.34 ± 0.09
T4	—	—	—	1.00	0.65 ± 0.09
T5	—	—	—	—	1.00

The strongest correlations involve the historical trust dimension T5, especially with T4 (0.65 ± 0.09) and T1 (0.64 ± 0.08), which is expected because T5 is an exponential history of recent trust evidence. Among the instantaneous dimensions T1–T4, all correlations remain at or below 0.41. Thus, T5 should be interpreted as a temporal memory channel rather than an independent sensor, while the instantaneous factors retain enough diversity to justify multi-factor fusion.

5 Conclusions

This study provides a reproducible trust-stream benchmark and a supervised multi-head risk model for UAV task allocation. On held-out seeds from the same generator, the model reaches 94.4% accuracy and a 91.8% F-score. A same-feature MLP reaches 94.7% and 92.1%; the paired differences are not significant. The principal empirical result is therefore that the chosen temporal summaries are informative for this generator, not that the proposed architecture is a better detector.

The 90.5% task-success value is a simulator-specific proxy. Since the allocation code applies MTIM-specific multipliers and does not use the learned incentive head, the 2.2 pp difference from MLP-Trust cannot support a causal claim about incentive learning. Likewise, the reported communication and latency values are generated rather than measured, and the code-level ablations do not isolate named architectural components. The short ns-3-derived check is the most useful warning: MTIM detects only two of five malicious nodes and does not outperform the simpler adapters.

The repository is nevertheless useful as an auditable starting point. A defensible next study should replace the synthetic allocation rules with a common policy for all detectors, measure actual runtime and messages, implement genuine agent-local observations and aggregation, and evaluate on multi-seed packet traces with attack parameters held out from training. Reinforcement-learning claims would additionally require environment interaction, reward-defined returns, and comparison with supervised and non-learning policies. Until those experiments are available, MTIM should be described as a supervised trust-stream proof of concept rather than a validated MARL or zero-trust deployment.

Author Contributions

All authors contributed to the conceptualization, methodology, analysis, and manuscript preparation. Author roles will be specified in the camera-ready version.

Funding

This research received no external funding.

Institutional Review Board Statement

Not applicable.

Informed Consent Statement

Not applicable.

Data Availability Statement

The replication package is included under **artifacts/**. It provides the code, configurations, trained models, evaluation outputs, statistical summaries, generated figures, baseline reimplementations, and packet-trace conversion materials needed to reproduce the trust-layer results reported in this manuscript. The included packet-trace validation is a single short sanity check rather than a multi-seed packet-level validation dataset.

Conflicts of Interest

The authors declare no conflict of interest.

References

- [1] Kundu, J.; Alam, S.; Das, J.C.; Dey, A.; Bari, S. Trust-Based Flying Ad Hoc Network: A Survey. *IEEE Access* **2024**, *12*, 99258–99281.
- [2] Noor, F.; Khan, M.A.; Al-Zahrani, A.; Ullah, I.; Al-Dhlan, K.A. A Review on Communications Perspective of Flying Ad-Hoc Networks: Key Enabling Wireless Technologies, Applications, Challenges and Open Research Topics. *Drones* **2020**, *4*, 65.
- [3] Brik, B.; Ksentini, A.; Bouaziz, M. Federated Learning for UAVs-Enabled Wireless Networks: Use Cases, Challenges, and Open Problems. *IEEE Netw.* **2020**, *34*, 144–150.
- [4] Nguyen, D.C.; Ding, M.; Pathirana, P.N.; Seneviratne, A.; Li, J.; Poor, H.V. Federated Learning for Internet of Things: A Comprehensive Survey. *IEEE Commun. Surv. Tutor.* **2021**, *23*, 1622–1658.
- [5] Qu, Y.; Dai, H.; Zhuang, Y.; Chen, J.; Dong, C.; Wu, F.; Guo, S. Decentralized Federated Learning for UAV Networks: Architecture, Challenges, and Opportunities. *IEEE Netw.* **2021**, *35*, 156–162.
- [6] Zuo, J.; Cao, R.; Qi, J.; Gao, P.; Wang, Z.; Li, J.; Zhang, L.; Lu, Y. A Hierarchical Blockchain-Based Trust Measurement Method for Drone Cluster Nodes. *Drones* **2023**, *7*, 627.
- [7] Ge, C.; Zhou, L.; Hancke, G.P.; Su, C. A Provenance-Aware Distributed Trust Model for Resilient Unmanned Aerial Vehicle Networks. *IEEE Internet Things J.* **2021**, *8*, 12481–12489.
- [8] Benfriha, S.; Labraoui, N.; Bensaid, R.; Bany Salameh, H.; Saidi, H. FUBA: A Fuzzy-Based Unmanned Aerial Vehicle Behaviour Analytics for Trust Management in Flying Ad-Hoc Networks. *IET Networks* **2024**, *13*, 208–220.
- [9] Li, H.; Li, X.; Dunkin, F.; Zhang, Z.; Lu, X. Adaptive Multi-Granularity Trust Management Scheme for UAV Visual Sensor Security under Adversarial Attacks. *Comput. Secur.* **2025**, *148*, 104108.
- [10] Akram, J.; Anaissi, A.; Rathore, R.S.; Jhaveri, R.H.; Akram, A. GALTrust: Generative Adversarial Learning-Based Framework for Trust Management in Spatial Crowdsourcing Drone Services. *IEEE Trans. Consum. Electron.* **2024**, *70*, 6196–6207.

- [11] Ceviz, O.; Sadioglu, P.; Sen, S.; Vassilakis, V.G. A Novel Federated Learning-Based IDS for Enhancing UAVs Privacy and Security. *Internet Things* **2025**, *31*, 101592.
- [12] Lu, Y.; Yang, T.; Zhao, C.; Chen, W.; Liu, Y.; Zeng, R. A Swarm Anomaly Detection Model for IoT UAVs Based on a Multi-Modal Denoising Autoencoder and Federated Learning. *Comput. Ind. Eng.* **2024**, *196*, 110454.
- [13] Gad, G.; Farrag, A.; Aboulfotouh, A.; Bedda, K.; Fadlullah, Z.M.; Fouda, M.M. Joint Self-Organizing Maps and Knowledge-Distillation-Based Communication-Efficient Federated Learning for Resource-Constrained UAV-IoT Systems. *IEEE Internet Things J.* **2024**, *11*, 34270–34282.
- [14] Wang, X.; Yuan, Q.; Liu, C.; Wang, P.; Tao, J.; Li, L.; Wang, X.; Wang, Y. Who Is the Wolf in Sheep's Clothing? A Context-Aware Trust Evaluation Model for Malicious UAV Detection during Authentication. *Comput. Netw.* **2026**, *279*, 112145.
- [15] Rose, S.; Borchert, O.; Mitchell, S.; Connelly, S. Zero Trust Architecture. *NIST Spec. Publ.* **2020**, *800-207*, 1–59.
- [16] Ekechi, C.C.; Elfouly, T.; Alouani, A.; Khattab, T. A Survey on UAV Control with Multi-Agent Reinforcement Learning. *Drones* **2025**, *9*, 484.
- [17] Akram, J.; Anaissi, A.; Rathore, R.S.; Jhaveri, R.H.; Akram, A. Digital Twin-Driven Trust Management in Open RAN-Based Spatial Crowdsourcing Drone Services. *IEEE Trans. Green Commun. Netw.* **2024**, *8*, 1061–1075.
- [18] Abubaker, Z.; Javaid, N.; Almogren, A.; Akbar, M.; Zuair, M.; Ben-Othman, J. Blockchain Service Provisioning and Malicious Node Detection via Federated Learning in Scalable Internet of Sensor Things Networks. *IEEE Trans. Netw. Serv. Manag.* **2023**, *20*, 1847–1865.
- [19] Akram, J.; Umair, M.; Jhaveri, R.H.; Riaz, M.; Chi, H.R.; Malebary, S. Chained-Drones: Blockchain-Based Privacy-Preserving Framework for Secure and Intelligent Service Provisioning in Internet of Drone Things. *Comput. Electr. Eng.* **2023**, *110*, 108772.
- [20] Zheng, J.; Xu, J.; Du, H.; Niyato, D.; Kang, J.; Nie, J.; Wang, Z. Trust Management of Tiny Federated Learning in Internet of Unmanned Aerial Vehicles. *IEEE Internet Things J.* **2024**, *11*, 21046–21060.
- [21] Tong, Z.; Wang, J.; Hou, X.; Chen, J.; Jiao, Z.; Liu, J. Blockchain-Based Trustworthy and Efficient Hierarchical Federated Learning for UAV-Enabled IoT Networks. *IEEE Internet Things J.* **2024**, *11*, 15504–15522.
- [22] Xu, Q.; Li, R.; Qi, Y.; Su, Z.; Fang, D. Trust-Enhanced Game Incentive for Secure Quantum Federated Learning in UAV-Assisted Wireless Networks. *IEEE J. Sel. Areas Commun.* **2025**, *43*, 2841–2856.
- [23] Hu, Y.; Gan, Y.; Wu, H.; Wang, C.; Ma, M.; Xiong, C. GDM-DTM: A Group Decision-Making-Enabled Dynamic Trust Management Method for Malicious Node Detection in Low-Altitude UAV Networks. *Sensors* **2025**, *25*, 3982.
- [24] Wang, X.; Yuan, Q.; Wang, Y.; Teng, J.; Tao, J.; Shen, M. Zero Trust-Driven Collaborative Intrusion Detection in Internet of Things: A Continuous Trust Assessment Approach. *IEEE Internet Comput.* **2025**, *29*, 37–47.
- [25] Al Ridhawi, I.; Aloqaily, M. Zero-Trust UAV-Enabled and DT-Supported 6G Networks. *IEEE Internet Comput.* **2023**, *27*, 59–67.
- [26] Wang, L.; Wang, K.; Pan, C.; Xu, W.; Aslam, N.; Hanzo, L. Multi-Agent Deep Reinforcement Learning Based Trajectory Planning for Multi-UAV Assisted Mobile Edge Computing. *IEEE Trans. Cogn. Commun. Netw.* **2021**, *7*, 73–84.
- [27] Nie, Y.; Zhao, J.; Gao, F.; Yu, F.R. Semi-Distributed Resource Management in UAV-Aided MEC Systems: A Multi-Agent Federated Reinforcement Learning Approach. *IEEE Trans. Veh. Technol.* **2021**, *70*, 13162–13173.

- [28] Alam, M.M.; Moh, S. Joint Optimization of Trajectory Control, Task Offloading, and Resource Allocation in Air-Ground Integrated Networks. *IEEE Internet Things J.* **2024**, *11*, 24273–24288.
- [29] Yang, H.; Zhao, J.; Xiong, Z.; Lam, K.-Y.; Sun, S.; Xiao, L. Privacy-Preserving Federated Learning for UAV-Enabled Networks: Learning-Based Joint Scheduling and Resource Management. *IEEE J. Sel. Areas Commun.* **2021**, *39*, 3144–3159.
- [30] Lim, W.Y.B.; Huang, J.; Xiong, Z.; Kang, J.; Niyato, D.; Hua, X.-S.; Leung, C.; Miao, C. Towards Federated Learning in UAV-Enabled Internet of Vehicles: A Multi-Dimensional Contract-Matching Approach. *IEEE Trans. Intell. Transp. Syst.* **2021**, *22*, 5140–5154.
- [31] Wang, S.; Hosseinalipour, S.; Gorlatova, M.; Brinton, C.G.; Chiang, M. UAV-Assisted Online Machine Learning Over Multi-Tiered Networks: A Hierarchical Nested Personalized Federated Learning Approach. *IEEE Internet Things J.* **2024**, *11*, 20899–20911.
- [32] Qureshi, K.I.; Wang, L.; Xiong, X.; Lodhi, M.A. Asynchronous Federated Learning for Resource Allocation in Software-Defined Internet of UAVs. *IEEE Internet Things J.* **2024**, *11*, 15523–15538.
- [33] Jing, Y.; Qu, Y.; Dong, C.; Ren, W.; Shen, Y.; Wu, Q.; Guo, S. Exploiting UAV for Air-Ground Integrated Federated Learning: A Joint UAV Location and Resource Optimization Approach. *IEEE Trans. Green Commun. Netw.* **2023**, *7*, 1420–1433.
- [34] Fu, Z.; Liu, J.; Mao, Y.; Qu, L.; Xie, L.F.; Wang, X. Energy-Efficient UAV-Assisted Federated Learning: Trajectory Optimization, Device Scheduling, and Resource Management. *IEEE Trans. Netw. Serv. Manag.* **2025**, *22*, 974–988.
- [35] Cao, X.; Nan, G.; Guo, H.; Mu, H.; Wang, L.; Lin, Y.; Zhou, Q.; Li, J.; Qin, B.; Cui, Q.; Tao, X.; Fang, H.; Du, H.; Quek, T.Q.S. Exploring LLM-Based Multi-Agent Situation Awareness for Zero-Trust Space-Air-Ground Integrated Network. *IEEE J. Sel. Areas Commun.* **2025**, *43*, 2230–2247.
- [36] Bertsekas, D.P.; Tsitsiklis, J.N. *Neuro-Dynamic Programming*; Athena Scientific: Nashua, NH, USA, 1996.
- [37] Oliehoek, F.A.; Spaan, M.T.; Vlassis, N. Optimal and Approximate Solutions to Decentralized POMDPs. In *Proceedings of the 17th International Conference on Automated Planning and Scheduling (ICAPS)*; AAAI Press: 2008; pp. 274–281.
- [38] Riley, G.F.; Henderson, T.R. The ns-3 Network Simulator. In *Modeling and Tools for Network Simulation*; Wehrle, K.; Günes, M.; Gross, J., Eds.; Springer: Berlin, Germany, 2010; pp. 15–34.
- [39] Hausknecht, M.; Stone, P. Deep Recurrent Q-Learning for Partially Observable MDPs. In *Proceedings of the AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*; AAAI Press: 2015; pp. 29–37.
- [40] Hausknecht, M.; Stone, P. Deep Reinforcement Learning in Parameterized Action Space. In *Proceedings of the International Conference on Learning Representations (ICLR)*; 2016.
- [41] Xiong, J.; Wang, Q.; Yang, Z.; Sun, P.; Han, L.; Zheng, Y.; Fu, H.; Zhang, T.; Liu, J.; Liu, H. Parametrized Deep Q-Networks Learning: Reinforcement Learning with Discrete-Continuous Hybrid Action Space. *arXiv* **2018**, arXiv:1810.06394.